

CLI Agent Semantic Validator Rules v0.1

Cross-object safety rules, red-line constraints, claim control, and implementation validation for C-Governed CLI Agent Mesh objects

Status	Draft normative profile v0.1
Date	2026-05-17
Package	C-Governed CLI Agent Mesh
Layer	c = a + b / SER / L4 / Agent Governance / Schema Extraction / Semantic Validation / Local Checker / Conformance / Witness
Document class	semantic-validator rules profile / implementation-readiness artifact / machine-facing package-control companion
Assertion class	C-A10 package-control artifact; C-A7 where witness, hash, evidence, canonicalization, or verification claims are made
Primary subject	persistent c entities and implementation workers validating that structurally valid CGAM objects do not smuggle unsafe authority, illegal action, cloud leakage, memory mutation, witness tampering, release claims, or conformance claims through valid-looking JSON.
Primary boundary	JSON Schema checks object shape. Semantic validators check meaning, cross-object consistency, authority flow, red-line violations, evidence sufficiency, and claim discipline. A structurally valid object is not automatically safe.

Kotov Ivan

Bruxelles, 2026

Contents

0. Executive definition	6
1. Purpose	6
2. Non-goals	7
3. Corpus bridge set	7
3.1 Explicit bridge: $c = a + b$	7
3.2 Quiet bridge I: logic	8
3.3 Quiet bridge II: cybernetics	8
3.4 Earth paragraph	8
4. Validation layers	8
4.1 Layer definitions	8
5. Validator result vocabulary	9
6. Universal red-line semantic rules	9
SVR-RED-001 — No red-line authorization	9
SVR-RED-002 — Red-line fields are const-false	10
SVR-RED-003 — Defensive label is not a bypass	10
7. Source availability and canonicity rules	10
SVR-SRC-001 — Source profile must exist before extraction	10
SVR-SRC-002 — Previously missing profiles now available	11
SVR-SRC-003 — Source hash required for extraction reports	11
SVR-SRC-004 — Duplicate source profile handling	11
8. Registry and schema-index rules	11
SVR-REG-001 — Object must exist in registry	11
SVR-REG-002 — Schema path must match registry	11
SVR-REG-003 — Status must not overstate extraction	11
SVR-REG-004 — Source-available is not extracted	12

9. Task / permission / registry alignment rules	12
SVR-TASK-001 — Task agent must be eligible	12
SVR-TASK-002 — Task scope must fit permission grant	12
SVR-TASK-003 — Permission cannot exceed registry ceiling	12
SVR-TASK-004 — Material task requires sandbox/worktree	13
SVR-TASK-005 — No task by vague prompt alone	13
10. AB / gate semantic rules	13
SVR-AB-001 — B alone is not authority	13
SVR-AB-002 — Dry-run must not mutate durable state	13
SVR-AB-003 — Local AB exception must be declared	13
SVR-AB-004 — Confirm phrase is not authorization	14
11. Sandbox / worktree rules	14
SVR-SBX-001 — Writes must stay inside allowed paths	14
SVR-SBX-002 — Denied paths override allowed paths	14
SVR-SBX-003 — Network default is deny unless explicitly granted	14
12. Witness / raw evidence / memory separation rules	14
SVR-WEV-001 — Witness is not raw evidence	14
SVR-RES-001 — Raw evidence requires sidecar	15
SVR-MEM-001 — Agent output is not memory	15
SVR-MEM-002 — Raw evidence is not memory content	15
SVR-MEM-003 — Immunity is not retaliation	15
13. Secrets / cloud data rules	16
SVR-SCDP-001 — Cloud context is a boundary crossing	16
SVR-SCDP-002 — Secrets are not prompt material	16
SVR-SCDP-003 — Synthetic fixtures must be clearly synthetic	16
SVR-SCDP-004 — Provider boundary is assumption, not proof	16
14. Review and quorum rules	16

SVR-REV-001 — Executor cannot be sole reviewer	16
SVR-REV-002 — Quorum is evidence, not sovereignty	16
SVR-REV-003 — Same-source consensus must be downgraded	17
SVR-REV-004 — Reviewer must have read access to required evidence	17
15. Incident and defensive emulation rules	17
SVR-IRP-001 — Preserve before repair	17
SVR-IRP-002 — Incident response is local defense	17
SVR-DEB-001 — Emulation stays in sandbox/garage	17
SVR-DEB-002 — Canary response must be bounded	18
16. Release / public surface rules	18
SVR-RPS-001 — File existence is not release	18
SVR-RPS-002 — Public bundle must not include restricted internals	18
SVR-RPS-003 — Release claims must match readiness gates	18
SVR-RPS-004 — SHA/PDF order matters	19
17. Conformance rules	19
SVR-CTM-001 — Conformance framework is not conformance execution	19
SVR-CTM-002 — Red-line failure blocks conformance claim	19
SVR-CTM-003 — Evidence packet must not embed raw evidence by default	19
SVR-CTM-004 — Safe fixtures only	19
18. Claim-control rules	20
SVR-CLAIM-001 — Draft documents cannot self-certify implementation	20
SVR-CLAIM-002 — Local checker pass is not conformance pass	20
SVR-CLAIM-003 — Schema extracted is not behavior proven	20
SVR-CLAIM-004 — Human/c gate required for high-risk surfaces	20
19. Rule severity classes	20
20. Machine-readable rule sketch	21
21. Required validator functions	21

22. Local checker binding	21
23. Conformance binding	22
24. Open issues	22
25. Acceptance checklist	23
26. Closing rule	23

0. Executive definition

CLI Agent Semantic Validator Rules define the non-structural checks required after JSON Schema validation and before any object is treated as operationally meaningful.

A JSON Schema can answer:

- Does the object have the required fields?
- Are field types and enum values valid?
- Does the object follow the expected shape?

A semantic validator answers:

- Does this object try to authorize something prohibited?
- Does the task match the permission grant?
- Does the permission grant exceed the agent registry ceiling?
- Does the sandbox actually cover all write paths?
- Does the witness event reference the correct transition?
- Does the memory gate attempt to promote raw evidence?
- Does the release claim exceed the readiness gate?
- Does a conformance result exist without evidence packets?
- Does a cloud context contain secret/private material?
- Does an executor review itself?

Compact formula:

- Schema validates the shape.
- Semantic validation validates the boundary.
- Conformance validates behavior.
- Authority remains with c and required human gates.

1. Purpose

The C-Governed CLI Agent Mesh package contains many machine-facing objects: task contracts, permission grants, handshakes, sandbox profiles, witness events, raw evidence sidecars, memory gates, review records, release records, conformance results, and data-boundary records.

Those objects can be structurally valid and still dangerous.

Example failure:

- A task contract may have all required fields but still request live external exploitation.

Example failure:

- A conformance result may be well-formed but claim CGAM-5 without evidence packets.

Example failure:

- A memory gate record may be valid JSON but attempt to promote raw incident evidence into c memory.

This profile defines the semantic validation layer that prevents those failures.

The validator exists to prevent:

1. structurally valid red-line behavior;
2. authority laundering through object references;

3. privilege drift between registry, permission, and task;
4. executor/reviewer collapse;
5. cloud leakage hidden behind redaction fields;
6. witness/evidence/memory conflation;
7. conformance washing;
8. release overclaiming;
9. local AB/gate inversions;
10. stale source-status claims;
11. unsafe fixture content;
12. public/restricted leakage.

2. Non-goals

This profile does not:

1. replace JSON Schema;
2. replace the local checker;
3. replace conformance execution;
4. replace human/c judgment;
5. replace legal, provider, or security review;
6. prove cryptographic soundness;
7. authorize task execution;
8. grant permissions;
9. publish releases;
10. promote memory;
11. certify operational safety;
12. make unsafe behavior safe by naming it “defensive”.

A semantic validator may block, warn, or route to review.

It may not become will, memory, judge, legal authority, release authority, incident authority, or sovereign decision-maker.

3. Corpus bridge set

3.1 Explicit bridge: $c = a + b$

Semantic validators belong to b: the technological substrate of procedures, checks, schemas, records, gates, and executable constraints.

They protect c.

They do not replace c.

They prevent components of b — including CLI agents, schemas, scripts, validators, checkers, fixtures, and package manifests — from silently becoming authority over c.

3.2 Quiet bridge I: logic

A schema can say an object is well-formed. It cannot decide whether the object is logically compatible with other objects. Semantic validation checks implication, contradiction, dependency, and forbidden combinations.

Example:

```
permission.network = false
but task.network_required = true
```

This is not a type error. It is a semantic contradiction.

3.3 Quiet bridge II: cybernetics

A control system fails when feedback is bypassed. Semantic validation enforces feedback loops: hold, deny, quarantine, freeze, witness, review, memory gate, release gate, and human gate. It prevents fast execution from outrunning governance.

3.4 Earth paragraph

The JSON Schema is the shape of the plug. Semantic validation checks whether the plug is connected to the right circuit. A perfectly shaped plug inserted into the wrong voltage still burns the machine. In this package, “valid JSON” is not enough. The validator must ask: does this task really fit this permission, this agent, this sandbox, this witness, this memory gate, and this release claim?

4. Validation layers

Semantic validation runs after structural validation.

Recommended order:

```
parse JSON/YAML
-> structural schema validation
-> canonical object registry check
-> source availability check
-> semantic single-object validation
-> semantic cross-object validation
-> red-line validation
-> release/conformance claim validation
-> local checker result
-> review / c gate / human gate where required
```

4.1 Layer definitions

Layer	Function	Failure default
SV-L0-parse	Input is readable JSON/YAML/Markdown-extracted object.	reject
SV-L1-schema	Object shape matches schema.	reject
SV-L2-registry	Object exists in Schema Object Registry.	hold_or_quarantine

Layer	Function	Failure default
SV-L3-source	Source profile exists and is canonical.	hold
SV-L4-single-object	Object meaning is internally coherent.	reject_or_hold
SV-L5-cross-object	Object is coherent with linked objects.	block_or_hold
SV-L6-red-line	No prohibited behavior is encoded.	red_line_fail
SV-L7-claim-control	Release/implementation/conformance claims are supported.	block_claim
SV-L8-gate-routing	Required c/human/review gates are present.	hold

5. Validator result vocabulary

Result	Meaning
pass	Object passes all applicable checks.
pass_with_warnings	Object may proceed only if warnings are non-material and recorded.
hold	Object is not rejected but cannot proceed until missing context is supplied.
block	Object cannot proceed in its current form.
quarantine	Object or output must be isolated for review.
freeze	Related operational path must stop before further action.
reject	Object is invalid for the intended use.
red_line_fail	Prohibited behavior or forbidden claim detected.
claim_blocked	Object may exist but may not support the asserted claim.
inconclusive	Validator cannot determine safety; material gates must fail closed.

Default rule:

```
UNKNOWN != allow.
UNKNOWN -> hold, block, or quarantine depending on risk.
```

6. Universal red-line semantic rules

These rules apply across all object families.

SVR-RED-001 — No red-line authorization

No object may authorize, request, imply, normalize, or launder:

```
hack-back
live external counter-operation
malware behavior
credential theft
covert persistence
evasion
unauthorized scanning
unauthorized exploitation
destructive external action
autonomous retaliation
```

- secret exfiltration
- cloud upload of secrets by default
- direct memory write by agents
- identity/core mutation by agents
- agent self-approval
- witness tampering
- release/publication without required gate

Failure result:

- red_line_fail

Required response:

- deny
- quarantine if object or output is contaminated
- freeze if an active path exists
- witness
- human/c review
- no automatic re-entry

SVR-RED-002 — Red-line fields are const-false

If a schema includes fields such as:

- hack_back_allowed
- autonomous_retaliation_allowed
- malware_behavior_allowed
- credential_theft_allowed
- direct_memory_write_allowed
- self_approval_allowed
- witness_tampering_allowed
- secret_prompting_allowed

they **MUST** be false.

Omission **MUST NOT** be interpreted as permission.

SVR-RED-003 — Defensive label is not a bypass

A task, fixture, emulation case, incident path, or immunity proposal cannot bypass red lines by using labels such as:

- defensive
- research
- simulation
- mirror
- canary
- garage
- incident
- emergency

The validator checks behavior, not label.

7. Source availability and canonicity rules

SVR-SRC-001 — Source profile must exist before extraction

Every object extracted into a schema must reference a canonical source profile.

If the source profile is missing:

- hold

No extraction by memory or guesswork.

SVR-SRC-002 — Previously missing profiles now available

The following profiles are now source-available in the working set:

```
CLI_Agent_Secrets_and_Cloud_Data_Policy_v0_1.md
CLI_Agent_Conformance_Test_Matrix_v0_1.md
CLI_Agent_Package_Index_and_Reading_Order_v0_1.md
```

The validator **MUST NOT** continue to mark SCDP or CTM object families as `blocked_source_missing` in a current registry based on the active working set.

SVR-SRC-003 — Source hash required for extraction reports

Before schema extraction or generated validator binding, the extraction report **SHOULD** record:

```
source_profile
source_path
source_sha256
source_date
source_role
```

If source hash is absent, implementation-ready may proceed only if the readiness gate explicitly records the gap as non-blocking for a draft handoff. Conformance-supported claims require source hashes.

SVR-SRC-004 — Duplicate source profile handling

If two canonical-looking files define the same object family, the validator **MUST**:

```
hold extraction
create discrepancy finding
require canonical source declaration
```

It **MUST NOT** silently merge both definitions.

8. Registry and schema-index rules

SVR-REG-001 — Object must exist in registry

Any object in `SCHEMA_INDEX.json`, examples, fixtures, checker outputs, or conformance results **MUST** exist in the Schema Object Registry.

Unknown object default:

```
hold_or_quarantine
```

SVR-REG-002 — Schema path must match registry

The object's schema file path **MUST** match the registry target path.

Mismatch result:

```
block
```

SVR-REG-003 — Status must not overstate extraction

Allowed progression:

```
prose_defined
```

- > schema_targeted
- > schema_extracted
- > example_bound
- > validator_bound
- > conformance_bound

A status cannot skip required evidence.

SVR-REG-004 — Source-available is not extracted

source_available means:

- source profile exists
- schema extraction still pending
- semantic validator binding still pending
- examples/conformance still pending unless separately recorded

It does not mean schema_extracted.

9. Task / permission / registry alignment rules

SVR-TASK-001 — Task agent must be eligible

A CLI_AGENT_TASK_CONTRACT.assigned_agent_ref must point to a registry entry whose status is eligible for the task.

Forbidden registry states for material tasks:

- unknown
- suspended
- quarantined
- revoked
- expired
- retired

Failure result:

- block

SVR-TASK-002 — Task scope must fit permission grant

A task contract may not request read, write, execute, network, cloud, memory, release, or incident capability beyond its permission grant.

If task scope exceeds permission:

- block

If uncertain:

- hold

SVR-TASK-003 — Permission cannot exceed registry ceiling

A permission grant may not exceed:

- agent trust ceiling
- auto-connect ceiling
- capability profile
- data boundary
- provider boundary
- revocation constraints

If permission exceeds registry ceiling:

```
block
witness
review
```

SVR-TASK-004 — Material task requires sandbox/worktree

Any task with material write, execute, release, memory, incident, schema extraction, or public-surface effect **MUST** reference a sandbox/worktree/execution boundary.

Missing sandbox result:

```
block
```

SVR-TASK-005 — No task by vague prompt alone

If an agent task is represented only by free text and lacks scope, permissions, outputs, failure behavior, data policy, and review path, it is not a valid material task.

Failure result:

```
hold_or_reject
```

10. AB / gate semantic rules

SVR-AB-001 — B alone is not authority

AB_MODE=B, apply=true, or enforce=true is insufficient to authorize material action.

The validator must also check:

```
task contract
permission grant
registry eligibility
sandbox/worktree
witness requirement
review gate
human/c gate where required
```

SVR-AB-002 — Dry-run must not mutate durable state

If dry_run=true or global mode is A/observe, durable mutation is forbidden.

Forbidden dry-run effects:

```
write to inbox
write to request queue
write to memory
write to release surface
publish
execute destructive command
rotate credential
modify protected branch
```

Failure result:

```
red_line_fail if material
block otherwise
```

SVR-AB-003 — Local AB exception must be declared

If any local module reverses or modifies global A/B meaning, it **MUST** have a local AB exception record.

Missing exception record result:

hold

SVR-AB-004 — Confirm phrase is not authorization

A confirm phrase may satisfy one gate.

It does not replace:

task contract
permission grant
review
witness
release gate
human anchor gate

11. Sandbox / worktree rules

SVR-SBX-001 — Writes must stay inside allowed paths

If an execution event, patch, diff, or output writes outside allowed paths, result is:

block_or_quarantine

If protected core paths are touched:

freeze
quarantine
witness
human/c review

SVR-SBX-002 — Denied paths override allowed paths

Denied paths always win over broad allowed paths.

Example:

```
allowed_paths: ["repo/"]  
denied_paths: ["repo/.git/", "repo/secrets/"]
```

A write to repo/secrets/ is prohibited.

SVR-SBX-003 — Network default is deny unless explicitly granted

No network access may be inferred from general task scope.

Network access requires:

explicit permission
purpose
allowed endpoints or abstract provider class
cloud data classification
logging/witness policy

12. Witness / raw evidence / memory separation rules

SVR-WEV-001 — Witness is not raw evidence

A witness event must not contain:

raw secrets

- private memory
- sealed material
- legal privileged content
- full prompt streams by default
- raw child data
- unredacted incident evidence
- agent self-narration as authority

If raw evidence appears in witness:

- quarantine
- redact
- create sidecar reference
- witness correction

SVR-RES-001 — Raw evidence requires sidecar

Raw evidence, if preserved, must be held in a restricted sidecar or equivalent custody object.

The validator must require:

- raw_artifact_ref
- hash
- custody class
- access policy
- retention class
- disclosure policy
- memory exclusion flag

SVR-MEM-001 — Agent output is not memory

Agent output may be proposed to memory.

It may not become memory without memory gate review.

Direct memory write by agent:

- red_line_fail

SVR-MEM-002 — Raw evidence is not memory content

Memory gate records must not promote raw evidence, secrets, unredacted incident data, or private material as ordinary memory.

Allowed alternatives:

- summary
- hash reference
- sidecar pointer
- redacted lesson
- uncertainty-marked operational note

SVR-MEM-003 — Immunity is not retaliation

Defensive immunity updates may update internal gates, signatures, deny rules, or fixtures.

They may not authorize retaliation, external action, malware behavior, or live counter-operation.

13. Secrets / cloud data rules

SVR-SCDP-001 — Cloud context is a boundary crossing

Any object sending data to a cloud agent must include or reference:

- data classification
- cloud context record
- redaction state
- provider boundary assumption
- allowed data classes
- forbidden data classes
- retention expectation
- witness requirement if sensitive

Missing classification result:

- hold

SVR-SCDP-002 — Secrets are not prompt material

Secrets, tokens, credentials, private keys, legal privileged material, incident evidence, private memory, identity material, and raw witness evidence must not enter cloud-agent context by default.

Failure result:

- freeze_and_escalate

SVR-SCDP-003 — Synthetic fixtures must be clearly synthetic

Any test object that resembles a secret must be clearly marked synthetic/fake.

If a fixture contains a plausible real credential pattern without synthetic marking:

- quarantine
- block test run

SVR-SCDP-004 — Provider boundary is assumption, not proof

Provider-specific privacy, retention, training, logging, or deletion claims require provider-current review.

Absent provider review:

- claim_blocked for provider-safe claims

14. Review and quorum rules

SVR-REV-001 — Executor cannot be sole reviewer

If the material executor is also the sole reviewer, validator result is:

- block

If the task is high-risk:

- red_line_fail or freeze depending on surface

SVR-REV-002 — Quorum is evidence, not sovereignty

A quorum record may support review.

It may not override:

- red-line rules
- c gate
- human gate
- memory gate
- release gate
- legal/security gate

SVR-REV-003 — Same-source consensus must be downgraded

If multiple reviewers share the same provider/runtime/source and no independent checker exists, consensus strength must be downgraded.

Failure to downgrade:

- warning_or_hold

SVR-REV-004 — Reviewer must have read access to required evidence

A review record is insufficient if the reviewer had no access to the necessary diff, witness ref, checker result, or evidence packet reference.

Result:

- hold

15. Incident and defensive emulation rules

SVR-IRP-001 — Preserve before repair

Incident repair tasks must show preservation or explicit non-applicability before repair.

Missing preservation for material incident:

- hold_or_freeze

SVR-IRP-002 — Incident response is local defense

Incident response may:

- stop
- freeze
- preserve
- quarantine
- revoke
- rotate owned credentials
- repair owned systems
- report through lawful route
- update internal defenses

It may not:

- hack back
- scan unauthorized systems
- attack external source
- steal credentials
- hide evidence

SVR-DEB-001 — Emulation stays in sandbox/garage

Defensive emulation must remain inside authorized isolated environments.

Any live external target, deployable exploit, or external counter-operation:

red_line_fail

SVR-DEB-002 — Canary response must be bounded

Canary response may record, signal internally, or route to incident handling.

It may not retaliate, deceive external systems beyond lawful owned channels, or create uncontrolled outbound behavior.

16. Release / public surface rules

SVR-RPS-001 — File existence is not release

A file is not release-valid merely because it exists.

Required checks:

canonical path
README/INDEX visibility
release notes alignment
public/restricted split
redaction review
hash/integrity record
local checker result
discoverability check
human/c gate where required

SVR-RPS-002 — Public bundle must not include restricted internals

Public bundles must not contain:

real secrets
private memory
raw evidence
incident-sensitive material
legal privileged material
real provider/account identifiers
real garage contents
canary values
unsafe operational details

Failure result:

freeze_and_redact

SVR-RPS-003 — Release claims must match readiness gates

A release surface record may not claim:

release_complete
implementation_ready
conformance_supported
public_release_ready

unless the matching gate records exist and pass.

Unsupported claim result:

claim_blocked

SVR-RPS-004 — SHA/PDF order matters

Semantic validator must reject release process claims where:

- SHA256SUMS generated before final artifacts
- PDF generated before Markdown freeze
- release notes not updated after artifact changes

Result:

- hold_public_surface

17. Conformance rules

SVR-CTM-001 — Conformance framework is not conformance execution

The presence of CTM, fixture pack, or validator rules does not support a claim that conformance has passed.

Conformance-supported requires:

- extracted schemas
- validated examples
- semantic validator results
- fixture run results
- evidence packets
- local checker result
- red-line failure handling proof

SVR-CTM-002 — Red-line failure blocks conformance claim

Any unresolved red-line failure blocks CGAM conformance claims.

Result:

- red_line_fail
- claim_blocked

SVR-CTM-003 — Evidence packet must not embed raw evidence by default

Evidence packets may include summary, hashes, witness refs, sidecar refs, and checker output.

They must not embed restricted raw evidence unless explicitly permitted by a restricted profile and access gate.

SVR-CTM-004 — Safe fixtures only

Conformance fixtures must be synthetic, local-only, non-operational, and non-abusive.

If fixture content becomes an abuse recipe:

- quarantine
- block_test_run
- redaction_review

18. Claim-control rules

SVR-CLAIM-001 — Draft documents cannot self-certify implementation

Markdown profiles may define intended behavior.

They do not prove implementation.

Forbidden unsupported claims:

```
implementation_ready
implementation_executed
conformance_supported
provider_safe
public_release_ready
```

without gate evidence.

SVR-CLAIM-002 — Local checker pass is not conformance pass

A local checker can support readiness.

It cannot by itself prove conformance.

SVR-CLAIM-003 — Schema extracted is not behavior proven

Extracted schemas support implementation.

They do not prove runtime enforcement.

SVR-CLAIM-004 — Human/c gate required for high-risk surfaces

Human/c gate is required for:

```
public release
archival release
DOI / Zenodo upload
release tag
public website publication
restricted-to-public promotion
high-risk implementation handoff
publication involving sensitive defensive material
publication after incident response
memory/core mutation
identity/core mutation
```

No agent, quorum, local checker, or validator may replace this gate.

19. Rule severity classes

Severity	Meaning	Default result
SV-S0-info	Informational; no action required.	pass
SV-S1-warning	Non-material issue; record and continue if allowed.	pass_with_warnings
SV-S2-hold	Missing context or unresolved dependency.	hold
SV-S3-block	Object cannot proceed in current form.	block
SV-S4-freeze	Active path must stop to preserve safety/evidence.	freeze

Severity	Meaning	Default result
SV-S5-red-line	Prohibited behavior or unsupported authority claim.	red_line_fail

20. Machine-readable rule sketch

A future extracted validator config may use:

```

schema_version: cli-agent-semantic-validator-rules-0.1
ruleset_id: cgam-semantic-validator-rules-v0-1
package: C-Governed CLI Agent Mesh
package_version: "0.1"
status: draft
rules:
  - rule_id: SVR-TASK-002
    title: Task scope must fit permission grant
    applies_to:
      - CLI_AGENT_TASK_CONTRACT
      - CLI_AGENT_PERMISSION_GRANT
    severity: SV-S3-block
    condition: task.scope exceeds permission.scope
    result: block
    witness_required: true

```

This YAML is a sketch, not the final implementation.

21. Required validator functions

Minimum P0 validator function targets:

Function	Scope
validate_registry_semantics	Registry status, eligibility, trust ceilings, source availability.
validate_ab_gate_semantics	A/B, dry-run, apply, confirm, arm flags, local exceptions.
validate_task_contract_semantics	Task scope, outputs, failure behavior, data policy.
validate_permission_grant_semantics	Permission bounds, duration, revocation, capability fit.
validate_sandbox_semantics	Allowed/denied paths, write/execute/network discipline.
validate_witness_semantics	Witness privacy, event family, transition linkage.
validate_raw_evidence_sidecar_semantics	Raw evidence custody, hash refs, access, retention.
validate_memory_gate_semantics	Memory promotion, evidence exclusion, uncertainty, immunity boundary.
validate_review_semantics	Executor/reviewer separation, quorum limits, conflict checks.
validate_secrets_cloud_data_semantics	Data class, cloud context, redaction, provider boundary.
validate_release_surface_semantics	Public/restricted split, artifact order, discoverability, claims.
validate_conformance_semantics	Test run, evidence packet, red-line failure, claim support.

22. Local checker binding

The local checker **MUST** call or emulate semantic validation for P0 object families before reporting pass for implementation-readiness-related checks.

Minimum required local checker output:

```

semantic_validation:
  ruleset: cli-agent-semantic-validator-rules-0.1
  status: pass | pass_with_warnings | hold | block | red_line_fail | inconclusive
  findings:
    - rule_id: SVR-TASK-002
      severity: SV-S3-block
      object_ref: task-contract/example-001
      result: block
      message: permission grant does not cover requested write path

```

If semantic validation is skipped, the local checker result for implementation-ready **MUST** be:

```
blocked_or_inconclusive
```

not pass.

23. Conformance binding

Conformance test results **MUST** include semantic validator results or a reasoned exception.

A conformance result without semantic validation is incomplete.

Required links:

```

CLI_AGENT_TEST_CASE
-> fixture / input object
-> structural schema result
-> semantic validator result
-> local checker result
-> evidence packet
-> conformance result

```

24. Open issues

ID	Issue	Required action
SVR-0I-001	Validator functions are specified but not implemented.	Extract rules into local checker / validator module.
SVR-0I-002	Schema-level \$defs and semantic rules must remain aligned.	Generate schema index and rule index together.
SVR-0I-003	Provider-specific validation requires current provider policy review.	Add provider profiles later.
SVR-0I-004	Retention classes still need global harmonization.	Keep conservative defaults or create retention/decay profile.
SVR-0I-005	Duplicate sandbox/worktree source variant must be resolved before extraction.	Declare canonical SWP source.
SVR-0I-006	Rule implementation language is not fixed.	Allow Python first; avoid tool-specific lock-in.
SVR-0I-007	Public examples must remain synthetic and non-operational.	Bind examples to fixture pack and public redaction profile.

25. Acceptance checklist

Before this profile can support implementation-ready handoff:

- [] P0 semantic rule families are accepted.
- [] SCDP, CTM, and Package Index are treated as source-available.
- [] Red-line rules are stable and shared across task, permission, incident, defensive, and release paths.
- [] Registry status classes include `source_available`.
- [] Local Checker Profile references semantic validation.
- [] Schema Object Registry aligns with this profile.
- [] JSON Schema Extraction Plan is patched or interpreted to include semantic validator binding.
- [] Fixture Pack contains only safe synthetic inputs.
- [] Release/Public Surface Profile controls public/restricted exposure.
- [] Conformance Test Matrix remains framework-only until evidence packets and runs exist.

26. Closing rule

Semantic validation is the boundary between a pretty object and a safe object.

Final rule:

```
Valid JSON is not valid authority.  
Valid schema is not valid behavior.  
Valid conformance shape is not conformance evidence.  
No semantic pass, no implementation claim.  
No evidence, no conformance claim.  
No gate, no authority.
```